

Tratamento de Exceções

Pré-requisitos: variáveis e tipos, funções, anotação de tipo, docstring, condicionais, laços de repetição, listas, dicionários, conjuntos, parâmetros posicionais/nomeados/default. Projeto de fundo: E-commerce simples.

1. Material explicativo

1.1 Erros de sintaxe vs. exceções

Os erros que podem ocorrer em um programa Python são de dois tipos:

Erros de sintaxe — causados por uma sintaxe incorreta no código. Provocam o encerramento do programa **antes mesmo dele começar a rodar**, pois o Python não consegue nem interpretar o código.

Exceções — ocorrem quando o programa está sintaticamente correto (o Python consegue interpretá-lo), mas o código resulta em um erro durante a **execução**.



```
python
valor = 10000

if valor > 2999
    print("Valor acima do limite.")

# SyntaxError: expected ':'
# (faltou o ":" no final do if — o programa nem chega a rodar)
```



```
python
valor = 10000
total = valor / 0
print(total)

# ZeroDivisionError: division by zero
# (o código está sintaticamente correto, mas o erro acontece ao executar)
```

1.2 O que é uma exceção

Uma condição de erro em tempo de execução é chamada de **exception** (exceção). Uma exceção pode ser:

Implícita — lançada automaticamente pelo próprio Python (ex.: dividir por zero, converter um texto inválido para número).

Definida por regra de negócio — lançada manualmente pelo programador, quando uma situação específica do problema que está sendo resolvido não deve ser permitida (ex.: um cliente sem crédito suficiente, um estoque insuficiente para uma venda).

Erros inesperados em tempo real de execução geralmente vêm de falhas na codificação, entrada de dados inconsistente, ou situações de negócio que não deveriam acontecer. Como não é possível antecipar 100% dos erros, planejar a captura deles é fundamental em qualquer aplicação que não pode simplesmente travar.

1.3 Blocos try / except

Para tratar uma exceção, colocamos o código que pode falhar dentro de um bloco `try:` e o tratamento do erro dentro de um bloco `except:`.

```
try:
    statements      # roda normalmente
except:
    statements      # roda se ocorrer uma exceção no try
else:
    statements      # roda somente se NÃO ocorrer nenhuma exceção
finally:
    statements      # roda sempre, com ou sem exceção
```

Quando ocorre um erro dentro do `try`, o Python interrompe imediatamente a execução daquele bloco e procura, na área de exceções (`except:`), o tratamento adequado para a falha.

```
python
x = 5
y = "hello"
z = x + y # provoca um TypeError

# TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
python
```

```
try:
    x = 5
    y = "hello"
    z = x + y
except TypeError:
    print("Ocorreu um erro, pois não é possível somar uma string com um inteiro")

# Ocorreu um erro, pois não é possível somar uma string com um inteiro
```

1.4 Diferentes tipos de exceções

Existem vários tipos de exceções pré-definidas no Python. As mais comuns são:

TypeError — uma operação é aplicada a um objeto de tipo errado (ex.: somar uma string com um inteiro).

NameError — um nome de variável ou função não é encontrado no escopo atual.

ValueError — uma função é chamada com um valor de tipo correto, mas inválido (ex.: `int("abc")`, que é uma string, mas não representa um número).

ZeroDivisionError — tentativa de dividir um número por zero.

KeyError — tentativa de acessar uma chave que não existe em um dicionário.

IndexError — tentativa de acessar uma posição que não existe em uma lista.

Todas essas exceções fazem parte de uma **hierarquia de classes** de erro do Python: toda exceção é uma classe, e essas classes herdam umas das outras (assim como `ZeroDivisionError` é um tipo mais específico de `ArithmeticError`). Não é preciso decorar essa árvore, mas é importante saber que ela existe — ela ajuda a entender, por exemplo, por que um `except Exception:` também captura um `ZeroDivisionError` (afinal, `ZeroDivisionError` é, por herança, também uma `Exception`). Esta é a hierarquia completa das exceções nativas do Python:



BaseException

+-- SystemExit

+-- KeyboardInterrupt

+-- GeneratorExit

+-- Exception

 +-- StopIteration

 +-- StopAsyncIteration

 +-- ArithmeticError

 +-- FloatingPointError

 +-- OverflowError

 +-- ZeroDivisionError

 +-- AssertionError

 +-- AttributeError

 +-- BufferError

 +-- EOFError

 +-- ImportError

 +-- ModuleNotFoundError

 +-- LookupError

 +-- IndexError

 +-- KeyError

 +-- MemoryError

 +-- NameError

 +-- UnboundLocalError

 +-- OSError

 +-- BlockingIOError

 +-- ChildProcessError

 +-- ConnectionError

 +-- BrokenPipeError

 +-- ConnectionAbortedError

 +-- ConnectionRefusedError

 +-- ConnectionResetError

 +-- FileExistsError

 +-- FileNotFoundError

 +-- InterruptedError

 +-- IsADirectoryError

 +-- NotADirectoryError

 +-- PermissionError

 +-- ProcessLookupError

 +-- TimeoutError

 +-- ReferenceError

 +-- RuntimeError

 +-- NotImplementedError

 +-- RecursionError

```
+-- SyntaxError
| +-- IndentationError
|   +-- TabError
+-- SystemError
+-- TypeError
+-- ValueError
| +-- UnicodeError
|   +-- UnicodeDecodeError
|   +-- UnicodeEncodeError
|   +-- UnicodeTranslateError
+-- Warning
    +-- DeprecationWarning
    +-- PendingDeprecationWarning
    +-- RuntimeWarning
    +-- SyntaxWarning
    +-- UserWarning
    +-- FutureWarning
    +-- ImportWarning
    +-- UnicodeWarning
    +-- BytesWarning
    +-- ResourceWarning
```

Repare que `BaseException` é a raiz de tudo, mas nós praticamente sempre tratamos (e criamos exceções personalizadas a partir de) `Exception` — as três exceções que ficam diretamente sob `BaseException` (`SystemExit`, `KeyboardInterrupt`, `GeneratorExit`) são casos especiais, ligados ao encerramento do próprio interpretador, e normalmente não devem ser capturados no dia a dia.

1.5 Capturando exceções específicas (múltiplos `except`)

Um `try` pode ter mais de uma cláusula `except`, uma para cada tipo de exceção que queremos tratar de forma diferente:

```
python
try:
    a = int(input('Informe um número: '))
    b = int(input('Informe um número: '))
    c = a / b
    print('Resultado da Divisão:', c)
except ZeroDivisionError:
    print("Não é possível fazer uma divisão por zero.")
except ValueError:
    print("O valor informado não é um número inteiro.")
```

1.6 Exceção genérica `Exception`

Se quisermos tratar qualquer outro erro "genérico" que não previmos especificamente, podemos usar `except Exception:`. Essa cláusula deve **sempre vir por último**, depois das exceções mais específicas — senão ela "rouba" o tratamento de todas as outras.

```
python
try:
    a = int(input('Informe um número: '))
    b = int(input('Informe um número: '))
    c = a / n          # "n" não foi definida em nenhum lugar
    print('Resultado da Divisão:', c)
except ZeroDivisionError:
    print('Não é possível fazer uma divisão por zero.')
except ValueError:
    print('O valor informado não é um número inteiro.')
except Exception:
    print('Um erro inesperado aconteceu.')

# Um erro inesperado aconteceu. (nesse caso, na verdade, um NameError)
```

1.7 Cláusula `else`

É possível completar um `try` com um bloco `else`, que só executa quando **nenhuma exceção** ocorre:

```
python
try:
    a = int(input('Informe um número: '))
    b = int(input('Informe um número: '))
    c = a / b
    print('Resultado da Divisão:', c)
except ZeroDivisionError:
    print('Não é possível fazer uma divisão por zero.')
except ValueError:
    print('O valor informado não é um número inteiro.')
else:
    print('Operação realizada com sucesso.')
```

1.8 Cláusula `finally`

O bloco `finally` é executado **em todas as situações**, ocorrendo ou não uma exceção. É muito usado para garantir que algo aconteça de qualquer forma (ex.: fechar um arquivo, liberar uma conexão,

imprimir um encerramento).

```
python
try:
    a = int(input('Informe um número: '))
    b = int(input('Informe um número: '))
    c = a / b
    print('Resultado da Divisão:', c)
except ZeroDivisionError:
    print('Não é possível fazer uma divisão por zero.')
except ValueError:
    print('O valor informado não é um número inteiro.')
else:
    print('Operação realizada com sucesso.')
finally:
    print('Final do Programa')
```

1.9 Apelidos (alias) de exceções: a cláusula `as`

Quando escrevemos `except TipoDeErro as apelido:`, estamos guardando o **objeto da exceção** em uma variável — um apelido — para poder usá-lo dentro do bloco `except` (por exemplo, para imprimir a mensagem do erro ou descobrir o tipo exato dele).

```
python
try:
    idade = int(input("Digite sua idade: "))
except ValueError as erro:
    print(f"Não foi possível converter o valor: {erro}")
```

Aqui, `erro` é apenas um apelido — poderia se chamar `e`, `erro_conversao`, `falha`, etc. O importante é que ele só existe **dentro** daquele bloco `except`; fora dele, a variável deixa de existir (o próprio Python a apaga automaticamente ao final do bloco).

Cuidado ao reaproveitar o mesmo apelido em `except` diferentes. É comum, por descuido, usar o mesmo nome de apelido em vários blocos `except` de um mesmo `try`, por exemplo tratando tipos de erro diferentes com o mesmo nome genérico `mensagem`:

```
python
```

```
except TypeError as mensagem:
    print(f'ERRO: {mensagem}')
except Exception as mensagem:
    print(f'ERRO Inesperado: {mensagem}')
```

Isso até funciona, mas **atrapalha a leitura**: como o mesmo nome `mensagem` aparece nos dois blocos, fica mais difícil perceber, à primeira vista, que cada `except` está lidando com um erro diferente. É melhor escolher um apelido que já diga a que exceção ele se refere — por exemplo, `TypeError as error_type` é melhor do que reaproveitar `mensagem`:



```
python
except TypeError as error_type:
    print(f'ERRO: {error_type}')
except Exception as error_generico:
    print(f'ERRO Inesperado: {error_generico}')
```

Assim, só de olhar o nome do apelido (`error_type`, `error_generico`) já entendemos qual exceção está sendo tratada em cada bloco, sem precisar checar o `except` correspondente.

1.10 Exemplos completos resolvidos

Os dois exemplos a seguir combinam tudo o que vimos até aqui (`try`, `except` múltiplo, apelidos, `else`, `finally`, `raise`) com conceitos que já conhecemos de aulas anteriores (`while`, `match/case`, funções).

Exemplo 1 — leitura validada com `else` e `raise`



```
python
```

```

while True:
    try:
        a = int(input('Digite um numero: '))
        b = int(input('Digite um outro numero: '))
        if a <= 0 or b <= 0:
            raise TypeError('O numero informado deve ser positivo')
        c = a / b
        print(f'Resultado: {c}')
    except ZeroDivisionError:
        print('ERRO: ocorreu uma divisão por zero')
    except ValueError:
        print('ERRO: O numero informado deve ser inteiro')
    except TypeError as error_type:
        print(f'ERRO: {error_type}')
    except Exception as error_generico:
        print(f'ERRO Inesperado: {error_generico}')
    else:
        # executa somente se não houve exceção
        print('Operação realizada com sucesso.')
        break
        # só sai do laço quando tudo deu certo
print('Fim do programa')

```

Note que o `break` está dentro do `else`: o programa só sai do laço `while True` quando a operação foi bem-sucedida (nenhuma exceção ocorreu). Se algo der errado, o `except` correspondente trata o erro e o `while` pede os números novamente. Note também que `TypeError` e `Exception` usam apelidos diferentes (`error_type` e `error_generico`), seguindo a boa prática vista na seção 1.9.

Exemplo 2 — calculadora com menu, `match`/`case` e `try` aninhado



python

```

def somar(a, b):
    return a + b

def subtrair(a, b):
    return a - b

def dividir(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print('Erro de divisão por zero.')

while True:
    try:
        print('1 - Soma')
        print('2 - Subtração')
        print('3 - Divisão')
        opcao = int(input('Escolha uma opção: '))
    except ValueError:
        print('O valor informado deve ser inteiro')
    else:
        try:
            a = int(input('Primeiro numero: '))
            b = int(input('Segundo numero: '))
        except ValueError:
            print('Os numeros informados devem ser inteiro')
        else:
            match opcao:
                case 1:
                    print(somar(a, b))
                case 2:
                    print(subtrair(a, b))
                case 3:
                    print(dividir(a, b))
                case _:
                    print('Opção inválida')
            break

```

Aqui temos um `try` dentro do `else` de outro `try`: primeiro validamos a opção do menu; só se a opção for válida (`else`) tentamos ler os dois números; só se os números forem válidos (`else` de novo) executamos a operação escolhida com `match`/`case`.

1.11 Instrução `raise` — gerando exceções manualmente

A instrução `raise` permite **forçar** a ocorrência de uma exceção específica, mesmo que o Python não a tivesse gerado sozinho. É muito usada para realizar **validações** de regras de negócio.



```
python
try:
    raise NameError      # gera uma exceção manualmente
except NameError:
    print("Ocorreu uma exceção")

# Ocorreu uma exceção
```



```
python
try:
    a = int(input('Informe um número: '))
    b = int(input('Informe um número: '))
    c = a / b
    if a < 0 or b < 0:
        raise TypeError    # gera TypeError se algum número for negativo
    print('Resultado da Divisão:', c)
except ZeroDivisionError:
    print('Não é possível fazer uma divisão por zero.')
except ValueError:
    print('O valor informado não é um número inteiro.')
except TypeError:
    print('O valor informado é negativo.')
```

Capturar vs. propagar. Quando uma exceção ocorre dentro de um `try` que tem um `except` correspondente, ela é **capturada e tratada** ali mesmo — o programa continua normalmente depois do bloco. Mas se dentro do `except` usarmos `raise` novamente (sem argumento algum, ou lançando uma nova exceção), a exceção é **propagada**: ela volta a subir, e quem chamou aquele trecho de código (uma função externa, por exemplo) é quem vai precisar tratá-la. Isso é útil quando uma função sabe que algo deu errado, mas quem decide o que fazer a respeito é quem chamou a função.

1.12 Recuperando informações do erro

Já vimos, na seção 1.9, que `except TipoDeErro as apelido:` guarda o objeto da exceção em uma variável. A partir desse objeto, podemos obter mais informações sobre o erro:

`apelido.__class__` — retorna a **classe** do erro.

`apelido.__class__.__name__` — retorna o **nome/código** do erro (como texto).

`str(apelido)` — retorna a **mensagem** do erro (também pode ser impressa diretamente com `print(apelido)` ou dentro de uma f-string, como `f'{apelido}'`).

```
python
titulo = "Valor Médio das Compras"
print(f'{titulo:^50}')
try:
    total_da_compra = float(input('Digite o valor total da compra: '))
    total_de_itens = int(input('Digite o total de itens: '))
    media = round(total_da_compra / total_de_itens)
    print('A média dos produtos é ', media)
except ZeroDivisionError as erro:
    print(f'{erro.__class__.__name__}: Divisao por zero')
except ValueError:
    print('Entre com um número')
```

1.13 Vantagens e desvantagens do tratamento de exceções

Vantagens:

Confiabilidade aprimorada — lidar com exceções adequadamente evita que o programa trave ou produza resultados incorretos por erros de entrada inesperados.

Código mais limpo — evita o uso de condicionais complexas só para verificar erros, deixando o código mais legível.

Manutenção facilitada — separa o código de tratamento de erros da lógica principal do programa.

Desvantagens:

Sobrecarga de desempenho — o interpretador precisa de trabalho extra para capturar e manipular a exceção.

Maior complexidade do código — tratar muitos tipos de exceção, ou implementar lógicas de erro complexas, pode deixar o código mais difícil de acompanhar.

1.14 Tema complementar: criando suas próprias exceções

Além das exceções prontas do Python (`ValueError`, `ZeroDivisionError` etc.), também podemos criar **exceções personalizadas**, específicas para a regra de negócio do nosso programa — por exemplo, `ClienteSemCreditoError` OU `EstoqueInsuficienteError`.

Para isso, criamos uma **classe** (assunto que veremos com profundidade mais adiante no curso) que herda de `Exception`. Por enquanto, basta usar sempre este "molde" pronto:

```
python
```

```
class NomeDaExcecao(Exception):  
    """Descreva aqui quando essa exceção deve ocorrer."""  
    pass
```

A partir daí, podemos usar `raise NomeDaExcecao("mensagem explicando o erro")` sempre que a situação de negócio ocorrer, e capturá-la com `except NomeDaExcecao as erro:` como qualquer outra exceção (usando um apelido, como vimos na seção 1.9).

```
python  
class InputError(Exception):  
    """Exceção lançada quando um valor de entrada não atende a uma regra do negócio."""  
    pass  
  
titulo = "Valor Médio das Compras"  
print(f'{titulo:^50}')  
  
total_da_compra = float(input('Digite o valor total da compra: '))  
total_de_itens = int(input('Digite o total de itens: '))  
try:  
    if total_de_itens <= 0:  
        raise InputError('Quantidade inválida')  
    media = round(total_da_compra / total_de_itens)  
    print('A média dos produtos é ', media)  
except InputError as erro:  
    print(f'{erro.__class__.__name__}: {erro} - Entre com valor maior que 0')  
except ValueError:  
    print('Entre com um número')
```

Uma exceção personalizada se comporta exatamente como qualquer exceção nativa: pode ser capturada com `except`, carregar uma mensagem própria e ser combinada com `else` e `finally` normalmente.

Exercícios complementares (exceções personalizadas)

Exercício complementar 1 Crie uma exceção `IdadeInvalidaError` e um pequeno programa que leia a idade de uma pessoa. Se a idade for menor que 0 ou maior que 120, lance a exceção com uma mensagem explicando o problema. Trate também o caso de a entrada não ser um número inteiro.

Exercício complementar 2 Crie uma exceção `SenhaFracaError` e um pequeno programa que leia uma senha. Se a senha tiver menos de 6 caracteres, lance a exceção com uma mensagem explicando a regra.

Exercício complementar 3 Crie uma exceção `NotaInvalidaError` e um pequeno programa que leia uma nota. Se a nota estiver fora da faixa de 0 a 10, lance a exceção. Trate também o caso de a entrada não ser um número.

2. Exercícios (dificuldade crescente)

Exercício 1 Leia um número inteiro digitado pelo usuário e imprima o seu quadrado (o número elevado ao quadrado). Trate `ValueError` caso o valor informado não seja um número inteiro.

Exercício 2 Elabore um algoritmo que leia dois números e imprima qual é o maior e qual é o menor. Se eles forem iguais, lance uma exceção explicando o motivo e trate essa exceção, encerrando a comparação de forma controlada (sem travar o programa).

Exercício 3 Elabore um algoritmo que leia uma letra entre A, B, C ou D e imprima o nome de uma fruta que comece com essa letra. Caso seja digitada qualquer outra letra, deverá ser lançada uma exceção.

Exercício 4 Para vários tributos, a base de cálculo é o salário mínimo. Elabore um algoritmo que leia o valor do salário mínimo e o valor do salário de uma pessoa. Calcule e imprima quantos salários mínimos essa pessoa ganha. Lance tratamentos de exceção condizentes (entrada inválida, salário mínimo negativo e divisão por zero).

Exercício 5 Escreva uma função `buscar_item_por_indice(lista: list[str], indice: int) -> str` que retorna o item de uma lista na posição informada. Em seguida, leia um índice digitado pelo usuário e mostre o produto correspondente em uma lista de produtos em promoção, tratando `IndexError` (posição que não existe) e `ValueError` (índice que não é um número inteiro).

Exercício 6 Crie um dicionário com o preço de alguns produtos (nome do produto -> preço). Peça ao usuário o nome de um produto e imprima o preço correspondente. Trate `KeyError` caso o produto não esteja cadastrado no dicionário.

Exercício 7 Peça dois números ao usuário e calcule a média entre eles. Use uma cláusula `else` para exibir uma mensagem de sucesso somente quando o cálculo for bem-sucedido, e uma cláusula `finally` para exibir sempre a mensagem "Cálculo finalizado.", tenha ocorrido erro ou não. Trate `ValueError` na conversão dos números.

Exercício 8 (nível avançado) Leia o nome de um produto, a quantidade disponível em estoque e a quantidade que um cliente deseja comprar. Se a quantidade desejada for maior que o estoque disponível, lance um `ValueError` explicando o problema. Separe a leitura dos dados (que pode gerar erro de conversão) da validação de negócio (estoque insuficiente) usando dois blocos `try` — um para cada situação — e apelidos (`as`) diferentes e descritivos para cada exceção, seguindo a boa prática vista na seção 1.9.

3. Implementação no projeto de fundo (E-commerce simples)

Enunciado: aplique tratamento de exceções ao projeto. **Sem alterar nenhuma das funções já existentes** (`gerar_codigo_produto`, `cadastrar_produto`, `buscar_produtos_por_termo`, `atualizar_estoque`,

`obter_todas_tags`, `buscar_produtos_por_tags`, `exibir_catalogo`), faça as seguintes melhorias:

Crie duas exceções personalizadas: `ProdutoNaoEncontradoError` e `EstoqueInsuficienteError`.

Crie uma nova função, `vender_produto`, que registra a venda de uma quantidade de um produto já cadastrado, buscando-o pelo nome:

Se o produto não for encontrado, lance `ProdutoNaoEncontradoError`.

Se a quantidade desejada for maior que o estoque disponível, lance `EstoqueInsuficienteError`.

Caso contrário, desconte a quantidade vendida do estoque do produto e retorne uma mensagem confirmando a venda.

No bloco `if __name__ == "__main__":`, demonstre o uso de `vender_produto` dentro de blocos

`try`/`except`/`else`/`finally`, cobrindo três cenários: venda bem-sucedida, produto inexistente e estoque insuficiente. Use um apelido diferente e descritivo para cada `except`, como visto na seção 1.9.